LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Branch: CSM
Section: B
Task No.: 2
Student Name: M. Niharika
Roll No.: A24126552097

---

1. TITLE

Design and Develop a Pure Semantic HTML Static Webpage Using Structural Tags, Tables for Layout, and Native Form Elements Without CSS

---

2. AIM

To design and develop a static webpage using pure semantic HTML5 elements — without any CSS — that demonstrates a header with navigation, a hero section, a two-column article/sidebar layout built with tables, native form controls, and a footer, relying entirely on browser default rendering.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand how a webpage can be structured using only semantic HTML tags.
- To use header, nav, section, main, article, aside, and footer to organise a document.
- To build a navigation bar using anchor links to in-page sections.
- To use a table for layout of a two-column content area without CSS.
- To create a form with fieldset, legend, labels, and various input types.
- To apply HTML5 form validation attributes such as required and placeholder.
- To observe how browsers render structural HTML using their own default styles.
- To verify that the page is readable and functional with no stylesheet at all.

---

4. THEORY

Semantic HTML refers to using HTML elements according to their intended meaning rather than only for their visual appearance. Tags such as header, nav, main, article, aside, and footer describe the role each block plays in the document, which helps browsers, search engines, and screen readers understand the page structure.

When no CSS is applied, the browser falls back to its own default stylesheet (the user-agent stylesheet). Headings, paragraphs, tables, and form controls are all rendered using these built-in defaults, which is why a page like this still looks organised even without any styling rules.

- **`header, nav, footer`** mark the top banner, the navigation links, and the closing section of the page.
- **`section, main, article, aside`** divide the body into a hero area, the primary content region, individual posts, and a side panel.
- **`<table> for layout`** arranges the header row and the two-column content area using rows and cells, a technique common before CSS layout existed.
- **`<code>`** marks inline text as source code, and browsers render it in a monospace font by default.
- **`<fieldset> and <legend>`** group related form controls together and give the group a visible caption.
- **`required and placeholder`** are HTML5 attributes that add built-in validation and hint text without any JavaScript.

The basic flow of the page is:

Browser requests HTML file → Browser applies its default stylesheet → Semantic structure is rendered as a readable page

---

5. ALGORITHM / PROCEDURE

1. Create a new HTML file named index.html.
2. Add the DOCTYPE declaration, the html tag, and a head section with a title.
3. Build a header containing the site name and a table-based navigation bar with anchor links.
4. Add a horizontal rule to visually separate the header from the page body.
5. Create a hero section with a centred heading, an introductory paragraph, and a call-to-action link.
6. Create a structural table with two columns to hold the main content and the sidebar.
7. In the left column, add a main element containing two article blocks with headings and paragraphs.
8. Inside the second article, build a form using fieldset, legend, labels, and input, select, and textarea controls.
9. In the right column, add an aside element listing benefits and required components using unordered and ordered lists.
10. Add a footer with a centred copyright notice.
11. Save the file and open it in a web browser without linking any CSS file.
12. Verify that the layout, navigation, and form controls render correctly using only browser defaults.

---

6. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>
```

```
    <!-- Header Section -->
    <header>
        <table width="100%" border="0" cellspacing="0" cellpadding="10">
            <tr>
                <td>
                    <h1>StaticWebpage</h1>
                </td>
                <td align="right">
                    <nav>
                        <a href="#home">Home</a>  | 
                        <a href="#articles">Articles</a>  | 
                        <a href="#about">About</a>  | 
                        <a href="#contact">Contact Us</a>
                    </nav>
                </td>
            </tr>
        </table>
    </header>

    <hr>

    <!-- Hero / Showcase Section -->
    <section id="home">
        <center>
            <h2>Semantic Structures Without CSS</h2>
            <p>This layout uses raw HTML components to build an organized document hierarchy that
browsers can cleanly parse out-of-the-box.</p>
            <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
        </center>
    </section>

    <hr>

    <!-- Main Content Layout using a Structural Table -->
    <table width="100%" border="0" cellspacing="0" cellpadding="20">
        <tr valign="top">

            <!-- Left Side: Main Content Column -->
            <td width="70%">
                <main id="articles">

                    <article>
                        <h2>1. Core Structural Tags</h2>
                        <p>Published: July 7, 2026</p>
                        <p>When you omit cascading style sheets entirely, the semantic markup
handles all of the structural heavy lifting. Elements like <code>&lt;main&gt;</code>,
<code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers and web
scrapers exactly how information flows.</p>
                        <p>Using raw browser defaults ensures excellent loading performance,
perfect accessibility compliance, and absolute structural clarity.</p>
                    </article>

                    <br><hr><br>

                    <article>
```

```
                        <h2>2. Native Interaction Elements</h2>
                        <p>Published: July 5, 2026</p>
                        <p>Browsers come equipped with excellent interactive elements by default.
We can capture text inputs, provide multiple-choice configurations, and submit structured
information to backend parsers using pure markup.</p>

                        <h3>Interactive Demonstration Forms</h3>
                        <form id="contact" action="#" method="post">
                            <fieldset>
                                <legend><b>User Inquiry Form</b></legend>
                                <p>
                                    <label for="name">Full Name: </label>
                                    <input type="text" id="name" name="username" required
placeholder="John Doe">
                                </p>
                                <p>
                                    <label for="email">Email Address: </label>
                                    <input type="email" id="email" name="useremail" required>
                                </p>
                                <p>
                                    <label for="topic">Inquiry Topic: </label>
                                    <select id="topic" name="usertopic">
                                        <option value="general">General Feedback</option>
                                        <option value="technical">Technical Architecture</option>
                                        <option value="academics">Academic Curriculum</option>
                                    </select>
                                </p>
                                <p>
                                    <label for="message">Message Details:</label><br>
                                    <textarea id="message" name="usermessage" rows="5" cols="40"
placeholder="Type your notes here..."></textarea>
                                </p>
                                <p>
                                    <input type="submit" value="Submit Form Data">
                                    <input type="reset" value="Clear Entries">
                                </p>
                            </fieldset>
                        </form>
                    </article>

                </main>
            </td>

            <!-- Right Side: Sidebar Column -->
            <td width="30%" bgcolor="#f4f4f4">
                <aside id="about">
                    <h3>Document References</h3>
                    <p>Static pages map data out directly without real-time database queries or
processor overhead.</p>

                    <h4>Core Benefits:</h4>
                    <ul>
                        <li>Instant browser painting</li>
                        <li>Low memory footprints</li>
                        <li>High data compatibility</li>
                    </ul>
```

```
                    <h4>Required Components:</h4>
                    <ol>
                        <li>Document Type Definition</li>
                        <li>Semantic Wrapper Blocks</li>
                        <li>Data Input Control Interfaces</li>
                    </ol>
                </aside>
            </td>

        </tr>
    </table>

    <hr>

    <!-- Footer Section -->
    <footer>
        <center>
            <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
        </center>
    </footer>

</body>
</html>
```

---

7. CODE EXPLANATION

Code	Explanation
<header>, <nav>, <footer>	Mark the top banner, the navigation links, and the closing section of the page.
<table> (layout)	Arranges the nav bar and the two-column body using rows and cells, without any CSS.
<section>, <main>, <article>, <aside>	Divide the body into a hero area, the primary content region, individual posts, and a side panel.
<a href="#id">	Creates an in-page link that jumps to the element with the matching id.
<code>	Displays inline text in a monospace font, typically used for code snippets.
<fieldset>, <legend>	Group related form controls and give the group a visible caption.
<label for="id">	Associates descriptive text with a form control that shares the same id.

required	HTML5 built-in validation that blocks submission if the field is left empty.
placeholder	Shows light hint text inside a field before the user types anything.
<select>, <option>	Create a dropdown list of choices for the Inquiry Topic field.
<textarea rows cols>	Provides a resizable multi-line text box for longer messages.
, ,	List the sidebar's Core Benefits and Required Components.

---

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each structural and interactive element was tested individually; the results were recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Page loads showing header, nav, hero, articles, sidebar, and footer	Page displayed correctly	Pass
TC-02	Click 'Articles' in the navigation bar	Browser jumps down to the Articles section	Jumped to correct section	Pass
TC-03	Leave the Full Name field empty and click Submit	Browser blocks submission and prompts for the field	Required-field prompt shown	Pass
TC-04	Enter an invalid email format	Browser shows an email-format validation error	Validation error displayed	Pass
TC-05	Open the Inquiry Topic dropdown	All three options are listed and selectable	Dropdown worked correctly	Pass
TC-06	Type multiple lines in Message Details	Textarea accepts and displays multi-line text	Multi-line text accepted	Pass

TC-07	Click 'Clear Entries'	All form fields reset to their default blank state	Fields cleared correctly	Pass

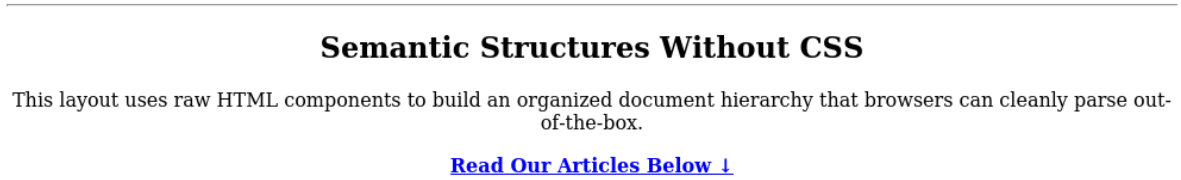
Additional Validation Test

Test Case	Input/Action	Expected Result	Status
TC-08	Resize the browser window	Table-based layout remains readable without a stylesheet	Pass
TC-09	View page with a screen reader	Semantic tags are announced in a logical reading order	Pass

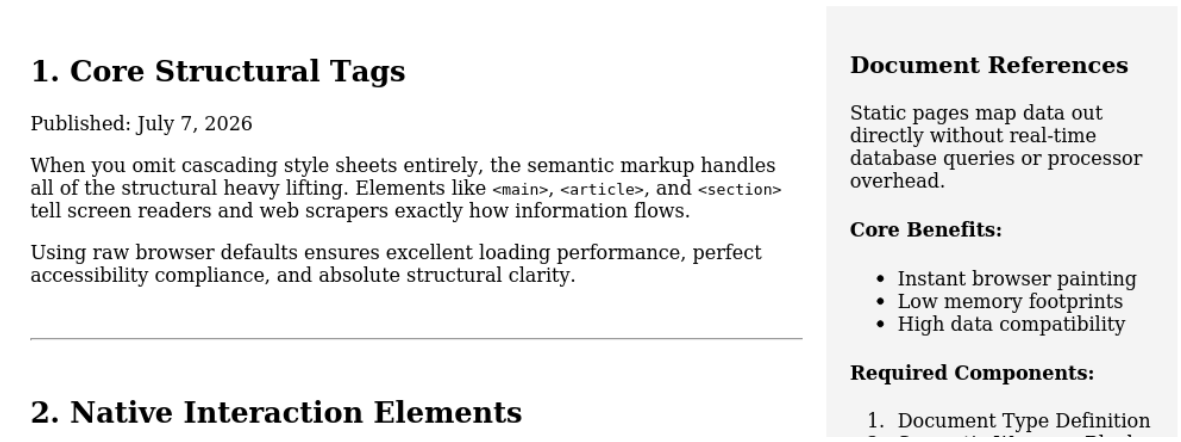
---

9. OUTPUT

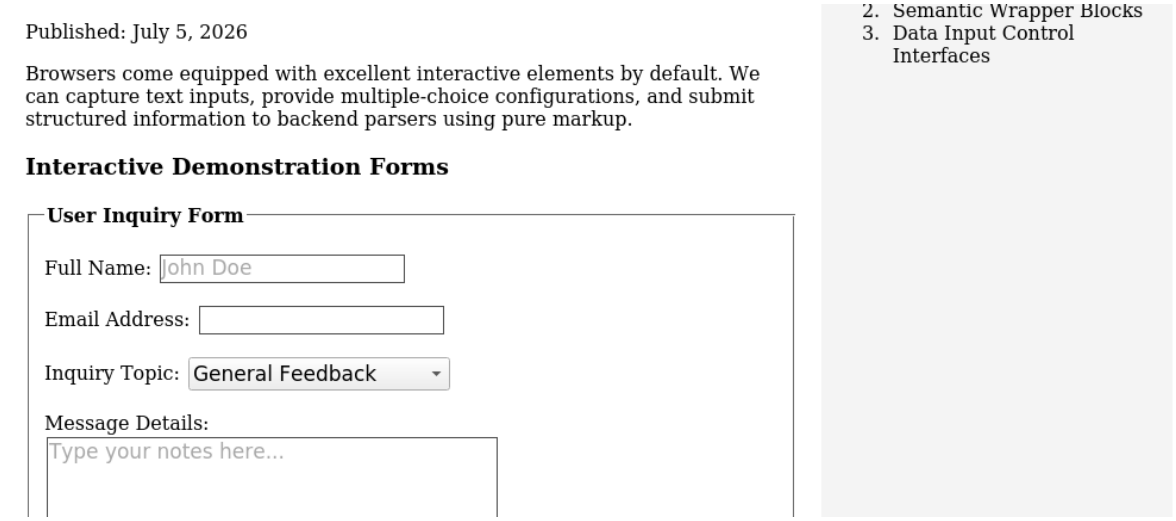
Output 1: Header, Navigation Bar, and Hero Section



Output 2: Article 1 – Core Structural Tags (with sidebar)



Output 3: Article 2 – Interactive User Inquiry Form



Output 4: Form Buttons, Sidebar Tail, and Footer

---

10. RESULT

Thus, a pure semantic HTML static webpage was successfully designed and developed without using any CSS. The header, navigation, hero section, table-based two-column layout, articles, interactive form, sidebar, and footer were implemented using semantic tags and were verified to render and function correctly using only the browser's default styling.